

An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

■ Date	@September 10, 2025
■ Description	Sequence modeling에서 RNN에 관한 연관성을 재고하고 새롭게 강력한 convolution 기반의 TCN(Temporal Convolution Network)을 제안
■ Keyword	Recurrent Networks Sequence Modeling TCN
■ Status	In progress
■ URL	https://arxiv.org/abs/1803.01271

Overview

지금까지는 sequence modeling task에서 RNN이 지배적으로 사용되었으나 Temporal Convolutional Network(TCN)을 제안해 convolution을 이용한 sequence modeling이 뛰어난 성능을 보인다는 것을 증명하고, 이를 LSTM이나 GRU와 같은 대표적인 순환 신경망과 비교하고 평가한다.

- Sequence modeling과 RNN의 일반적인 연관성을 재고
- TCN을 제안해 더 효과적인 sequence modeling을 달성

Introduction

Sequence processing과 recurrent network과의 연관성을 다양한 sequence modeling task에서 convolutional architecture와 recurrent architecture를 비교함으로써 다시 고려해보도록 한다.

다양한 sequence modeling task에서 TCN의 성능이 baseline(recurrent architecture)를 능가했다.

Recurrent architecture의 가장 큰 특징인 무한하게 긴 history를 포착할 수 있다는 점도 TCN에서 더 뚜렷하게 보인다는 것을 증명한다.

Methods: Temporal Convolutional Networks

TCN 구조의 가장 큰 특징 2가지는

1. Casual convolution을 사용해 미래에서 과거로의 정보 leakage가 발생하지 않는다.
2. 이 구조는 모든 길이의 sequence를 input으로 가질 수 있고 동일한 길이의 output sequence를 도출한다.

TCN은 심플함과 autoregressive prediction, 매우 긴 메모리라는 특징을 한번에 가진다.

Sequence Modeling

Sequence modeling task의 가장 큰 핵심은 과거에 발생한 input들만 가지고 output을 예측해야한다는 점이다.

이러한 점은 주로 autoregressive prediction(target output은 input이 time step 1만큼 이 동한것)으로 구현되어왔지만 machine translation이나 sequence-to-sequence와 같은 경우에는 모든 input sequence를 활용해 예측할 수 있다.

Casual Convolutions

TCN의 기본 principles을 구현하기 위한 방법으로

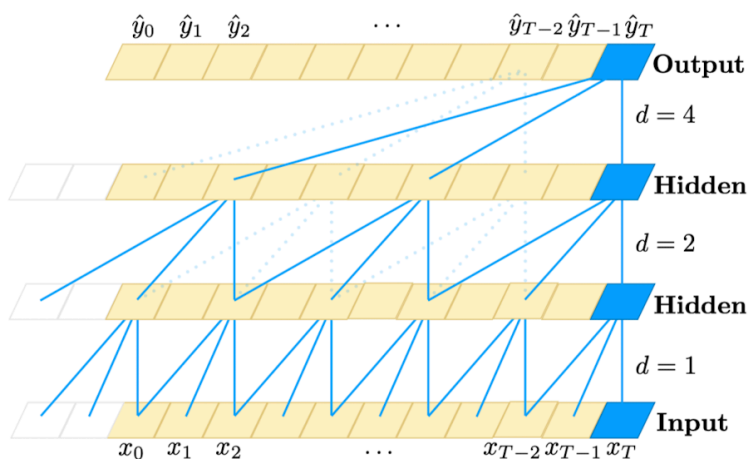
1. input의 길이와 동일한 길이의 output: 1D fully-convolution network architecture를 사용
모든 hidden layer의 길이는 input layer의 길이와 같도록 (kernel size -1)로 zero padding된다.
2. 정보 leakage가 발생하지 않는다: casual convolution 사용

⇒ TCN = 1D FCN + casual convolutions

이러한 디자인의 가장 큰 단점은 매우 긴 effective history size를 가지기 위해서는 매우 깊은 network를 사용하거나 매우 큰 filter를 사용해야한다.

이러한 점을 해결하기 위해 Dilated Convolution을 사용한다.

Dilated Convolutions



Casual convolution은 effective history size가 네트워크 깊이에 따라 선형적으로 증가하는데 dilated convolution은 지수적으로 증가한다.

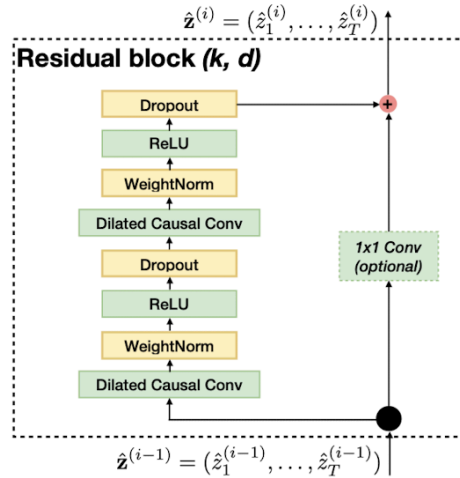
$$F(s) = (\mathbf{x} *_{d} f)(s) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}_{s-d \cdot i}$$

여기서 d 가 dilation factor이다. 큰 dilation을 사용할수록 상위층의 output은 input의 더 넓은 범위를 의미한다. → ConvNet의 receptive field를 확장하는 것과 같은 효과

이러한 경우에 해당 layer의 effective history는 $(k - 1)d$ 가 된다.

이로써 receptive field를 늘리는 방법이 2가지가 존재한다. 1. filter size를 키우거나 2. dilation size를 키우거나

Residual Connection



TCN의 residual connection 구조의 경우 input과 output이 다른 width를 가질 수 있는데 이를 보완하기 위해 추가적인 1x1 convolution을 사용한다.

$$o = \text{Activation}(x + F(x))$$

Discussion

TCN의 장점

- **Parallelism:** convolution 연산은 병렬적으로 처리가 가능하다.
- **Flexible receptive field size:** 여러개의 casual dilated convolutional layer를 쌓거나 filter size를 키우거나, dilation factor를 키우거나 등 다양한 방법으로 receptive field size를 조정할 수 있다.
- **Stable gradients:** recurrent architecture와 달리 sequence의 temporal direction과 다른 backpropagation path를 가지기 때문에 gradient exploding/vanishing 문제가 발생하지 않는다.
- **Low memory requirement for training:** backpropagation path가 network depth에만 의존되어서 RNN보다 적은 memory를 사용한다. → Training 단계에서만 이점
- **Variable length inputs:** input sequence의 길이가 길어지면 필터가 슬라이딩하는 횟수만 늘어나고, 짧아지면 줄어든 뿐 구조의 변환이 발생하지 않는다.

TCN의 단점

- **Data storage during evaluation:** RNN은 과거 정보를 유지하기 위해 hidden state 값만 가지면 되는데 TCN은 effective history 길이만큼을 가져야 한다. → Evaluation 단계에서만 이점

추론하기 위해 가져야하는 메모리의 양이 $RNN > TCN$

- **Potential parameter change for a transfer of domain:** 다른 domain들은 서로 모델이 얼마만큼의 history를 기억해야하는지 요구하는 정도가 다르기 때문에 memory 활용의 차이가 발생하고 충분한 receptive field를 가지지 못하면 안 좋은 성능을 보일 수 있다.

Experiments

Sequence Modeling Tasks

Sequence Modeling Task	Model Size ($\approx$)	Models			
		LSTM	GRU	RNN	TCN
Seq. MNIST (accuracy ^h)	70K	87.2	96.2	21.5	99.0
Permuted MNIST (accuracy)	70K	85.7	87.3	25.3	97.2
Adding problem $T=600$ (loss ^ℓ)	70K	0.164	5.3e-5	0.177	5.8e-5
Copy memory $T=1000$ (loss)	16K	0.0204	0.0197	0.0202	3.5e-5
Music JSB Chorales (loss)	300K	8.45	8.43	8.91	8.10
Music Nottingham (loss)	1M	3.29	3.46	4.05	3.07
Word-level PTB (perplexity ^ℓ)	13M	78.93	92.48	114.50	88.68
Word-level Wiki-103 (perplexity)	-	48.4	-	-	45.19
Word-level LAMBADA (perplexity)	-	4186	-	14725	1279
Char-level PTB (bpc ^ℓ)	3M	1.36	1.37	1.48	1.31
Char-level text8 (bpc)	5M	1.50	1.53	1.69	1.45

Synthetic Stress Test: RNN에서 가장 많이 사용되던 데이터셋

- **The adding problem:** 두번째 차원이 1로 되어있는 두 값의 합을 예측하는 task
- **Sequential MNIST and P-MNIST:** MNIST 이미지가 숫자 분류를 위해 sequence로 제공
- **Copy memory:** 다양한 길이의 시퀀스에 걸쳐 정보를 얼마나 오래 유지할 수 있는 지 확인

⇒ **TCN이 더 효과적으로 장기기억을 다룸**

폴리포닉 음악 모델링

- **JSB Chorales and Nottingham**

Language Modeling

- Penn Treebank
- Wikitext-103
- LAMBADA
- text8

⇒ word-level PTB를 제외하고는 모두 TCN이 더 좋은 성능을 보임

Memory Size of TCN and RNNs

이론적으로 recurrent architecture의 가장 큰 장점은 unlimited memory이다.

하지만 copy memory task를 통해 분석해본 결과 TCN이 recurrent architecture보다 더 긴 effective history를 유지할 수 있었다.

Summary

우리는 시퀀스 모델링(sequence modeling)에서 순환 신경망(recurrent networks, RNNs)이 일반적으로 선호되는 방식임에도 불구하고, 합성곱 신경망(convolutional networks, CNNs) 기반의 모델이 다양한 태스크에서 RNN보다 우수한 성능을 보일 수 있음을 체계적으로 평가합니다.

기존의 RNN 벤치마킹에 자주 사용되는 광범위한 시퀀스 모델링 태스크 및 데이터셋에서 간단한 Temporal Convolutional Network (TCN) 아키텍처와 LSTM(Long Short-Term Memory), GRU(Gated Recurrent Unit)와 같은 대표적인 RNN 아키텍처들을 비교 분석합니다.